Classificação de clientes

Naive Bayes, árvore de decisão e regressão logística

Prof. Dr. Márcio Andrey Teixeira
IFSP Campus Catanduva

Análise do notebook e interpretação dos resultados

O que o classificador aprende

Entrada: renda anual e pontuação de gasto de um cliente.

Saída: uma das classes 0, 1, 2, 3 ou 4.

O K-means anterior produziu os rótulos da coluna Cluster.

Os classificadores aprendem a reproduzir esses rótulos.

A acurácia mede concordância com os clusters, não qualidade comercial da segmentação.

Estrutura do dataset

Coluna	Papel nesta implementação
CustomerID	Identificador, excluído das entradas
Genre e Age	Disponíveis, mas não utilizados
Annual Income (k$)	Entrada: renda anual em milhares de dólares
Spending Score (1-100)	Entrada: pontuação de gasto
Cluster	Alvo: código do agrupamento anterior

200 registros, 6 colunas e nenhum valor ausente nas saídas salvas.

Leitura e exploração dos dados

```
drive.mount('/content/drive')
dataset = pd.read_csv(caminho_do_csv)
dataset.head()
dataset.info()
dataset.describe()
dataset.isnull().sum()
```

head: amostra inicial info: tipos e preenchimento
describe: estatísticas isnull: valores ausentes

Atributos de entrada e rótulo

```
X = dataset[['Annual Income (k$)',
             'Spending Score (1-100)']].values
y = dataset['Cluster'].values
```

Renda	Pontuação	Cluster
15	39	4
15	81	2
16	6	4
16	77	2
17	40	4

X: (200, 2)
y: (200,)

Divisão em treinamento e teste

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2,
    random_state=42, stratify=y
)
```

Conjunto	Quantidade	Uso
Treinamento	160 clientes	Ajustar os modelos
Teste	40 clientes	Medir resultados fora do treino

Classes no teste: 0 = 16, 1 = 8, 2 = 4, 3 = 7, 4 = 5.

Gaussian Naive Bayes

Estima a probabilidade de cada classe a partir dos atributos.

Supõe independência condicional entre atributos dada a classe.

GaussianNB modela cada atributo por uma distribuição gaussiana.

Escolhe a classe com maior probabilidade estimada.

Correção do notebook: Naive Bayes não usa a regra dos vizinhos mais próximos.

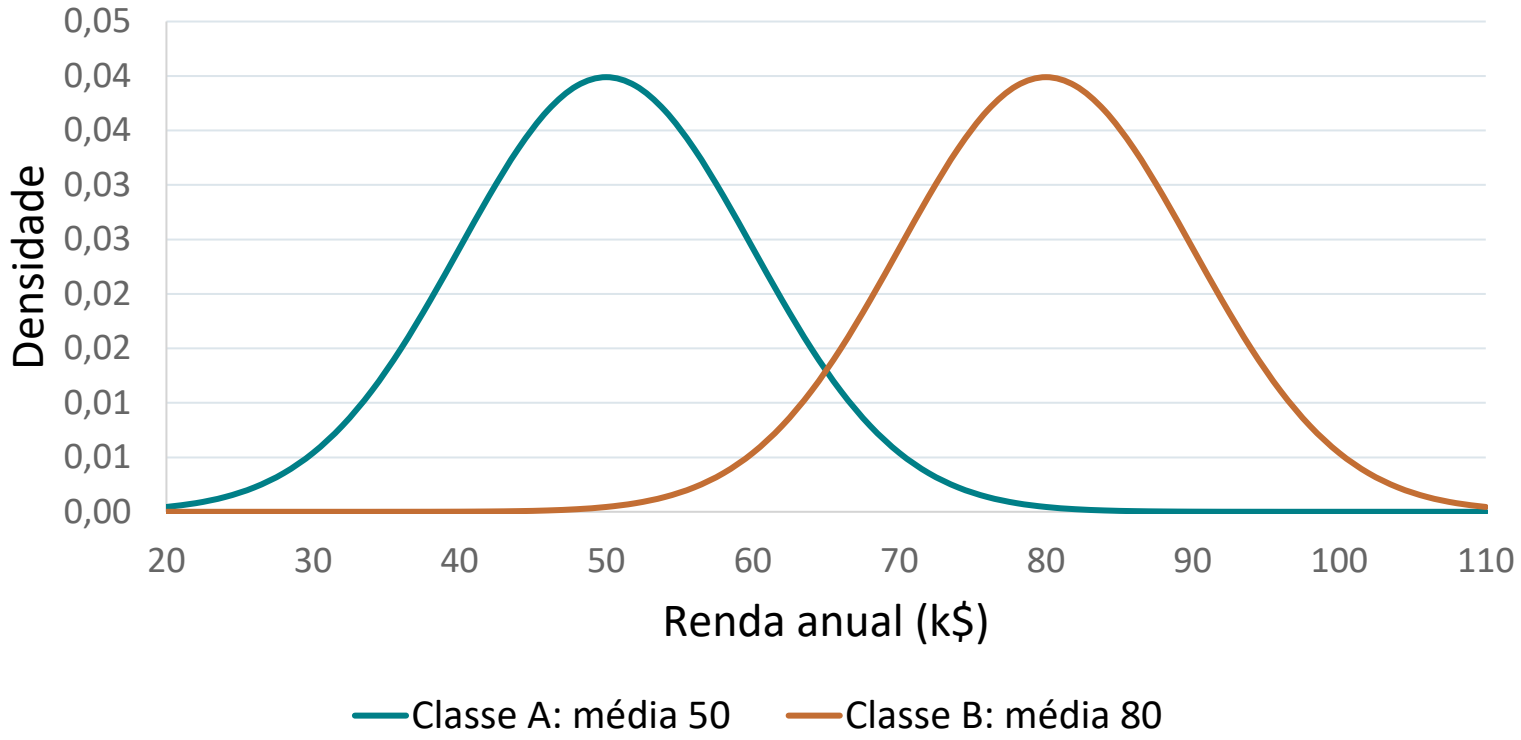
O teorema de Bayes

$$P(C \text{ dado } x) = P(C) \times f(x \text{ dado } C) / f(x)$$

Termo	Significado
P(C)	Probabilidade da classe antes de observar o cliente
f(x dado C)	Compatibilidade dos atributos com a classe
P(C dado x)	Probabilidade da classe após observar os atributos
f(x)	Fator comum que normaliza as pontuações

Para escolher a maior posterior, basta comparar os numeradores de cada classe.

A distribuição gaussiana de um atributo



A média define o centro.

O desvio padrão define a dispersão.

Um valor próximo da média tem maior densidade.

Curvas ilustrativas. O modelo ajusta uma gaussiana por atributo e por classe.

A ligação com predict e predict_proba

```
novo_cliente = np.array([[60, 70]])
novo_scaled = scaler.transform(novo_cliente)

nb_model.predict(novo_scaled)
nb_model.predict_proba(novo_scaled)
nb_model.classes_
```

predict devolve a classe escolhida.
predict_proba devolve uma probabilidade por classe.

Usar o scaler do treino. Este trecho não foi executado e não reproduz o exemplo fictício.

Vantagens e limites do GaussianNB

Treinamento simples: frequências, médias e variâncias.

Atributos correlacionados podem contar evidências semelhantes mais de uma vez.

Padronizar não transforma automaticamente uma distribuição em gaussiana.

Boa classificação não garante probabilidades bem calibradas.

No notebook, o modelo tem cinco classes. O cálculo segue a mesma lógica do exemplo.

Treinamento do Naive Bayes

```
nb_model = GaussianNB()
nb_model.fit(X_train_scaled, y_train)
y_pred_nb = nb_model.predict(X_test_scaled)

accuracy_score(y_test, y_pred_nb)
classification_report(y_test, y_pred_nb)
```

Resultado salvo: 40 acertos em 40 clientes

Árvore de decisão

```
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)
y_pred_dt = dt.predict(X_test)
```

Aprende sequências de testes sobre renda e pontuação.

Usa os valores originais, sem StandardScaler.

Não há limite de profundidade definido no código.

Resultado salvo: 38 acertos e 2 erros. A estrutura da árvore não aparece no notebook.

Regressão logística multiclasse

```
lr = LogisticRegression(max_iter=1000, random_state=42)
lr.fit(X_train_scaled, y_train)
y_pred_lr = lr.predict(X_test_scaled)
```

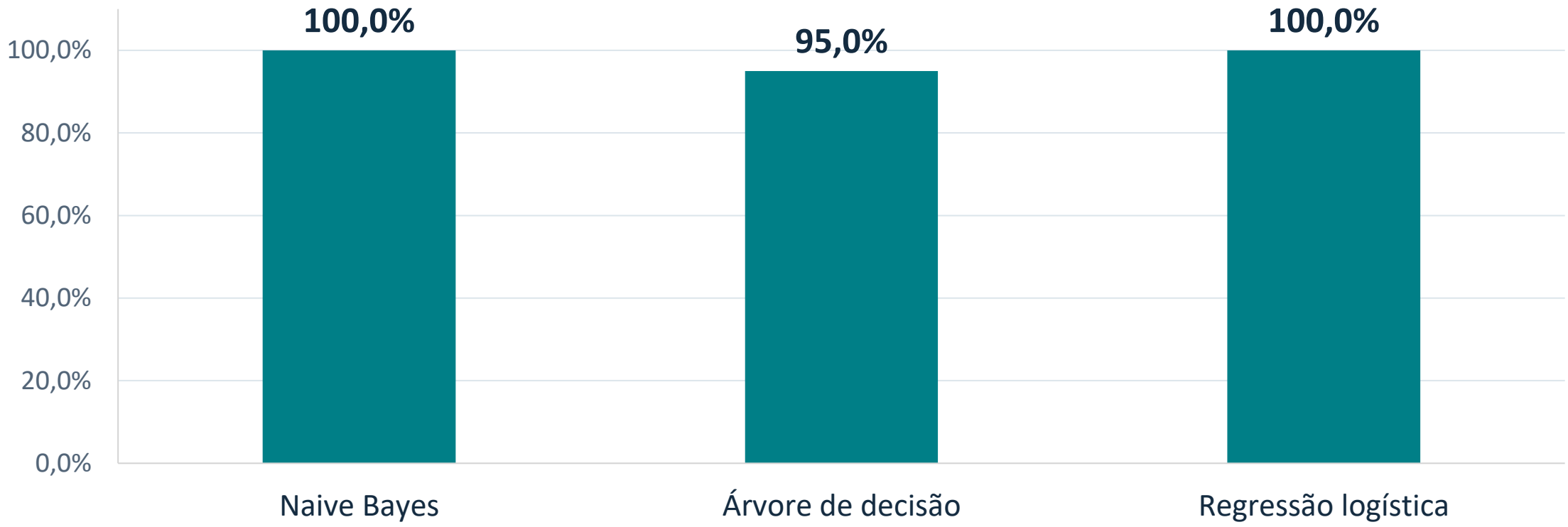
Estima probabilidades e escolhe uma das cinco classes.

max_iter=1000 aumenta o limite de iterações do ajuste.

Recebe os mesmos dados padronizados do Naive Bayes.

Resultado salvo: 40 acertos em 40 clientes.

Comparação dos resultados no teste



40 exemplos no teste. Uma amostra representa 2,5 pontos percentuais de acurácia.

Leitura da matriz de confusão

Linha: classe real registrada em y_test.

Coluna: classe prevista pelo modelo.

Diagonal principal: classificações corretas.

Fora da diagonal: erros e seu destino.

A posição [1, 0] registra clientes da classe 1 previstos como classe 0.

Matriz de confusão do Naive Bayes

Classe prevista

Real / Prev.	0	1	2	3	4
0	16	0	0	0	0
1	0	8	0	0	0
2	0	0	4	0	0
3	0	0	0	7	0
4	0	0	0	0	5

40 acertos

0 erros

Acurácia = 100%

Resultados salvos no notebook. Linhas = classes reais. Colunas = previsões.

Matriz de confusão da árvore

Classe prevista

Real / Prev.	0	1	2	3	4
0	16	0	0	0	0
1	1	7	0	0	0
2	0	0	3	0	1
3	0	0	0	7	0
4	0	0	0	0	5

Classe 1: um cliente previsto como 0

Classe 2: um cliente previsto como 4

38 / 40 = 95%

Resultados salvos no notebook. Linhas = classes reais. Colunas = previsões.

Matriz da regressão logística

Classe prevista

Real / Prev.	0	1	2	3	4
0	16	0	0	0	0
1	0	8	0	0	0
2	0	0	4	0	0
3	0	0	0	7	0
4	0	0	0	0	5

40 acertos

0 erros

Mesmo resultado do Naive Bayes no teste

Resultados salvos no notebook. Linhas = classes reais. Colunas = previsões.

Precisão, recall e F1

Métrica	Pergunta ou cálculo
Acurácia	Acertos totais / total de exemplos
Precisão	TP / (TP + FP): quantos previstos na classe estão corretos?
Recall	TP / (TP + FN): quantos reais da classe foram encontrados?
F1	2 × precisão × recall / (precisão + recall)
Support	Quantidade real de exemplos da classe

Cálculo das métricas da classe 2

Existem 4 clientes reais da classe 2. A árvore encontra 3.

TP = 3, FP = 0 e FN = 1.

Precisão = 3 / 3 = 100%

Recall = 3 / 4 = 75%

F1 = 2 × 1 × 0,75 / 1,75 ≈ 85,71%

Relatório de classificação da árvore

Classe	Precisão	Recall	F1	Support
0	0,94	1,00	0,97	16
1	1,00	0,88	0,93	8
2	1,00	0,75	0,86	4
3	1,00	1,00	1,00	7
4	0,83	1,00	0,91	5

NB e regressão logística: precisão, recall e F1 iguais a 1,00 em todas as classes neste teste.

Médias macro e ponderada

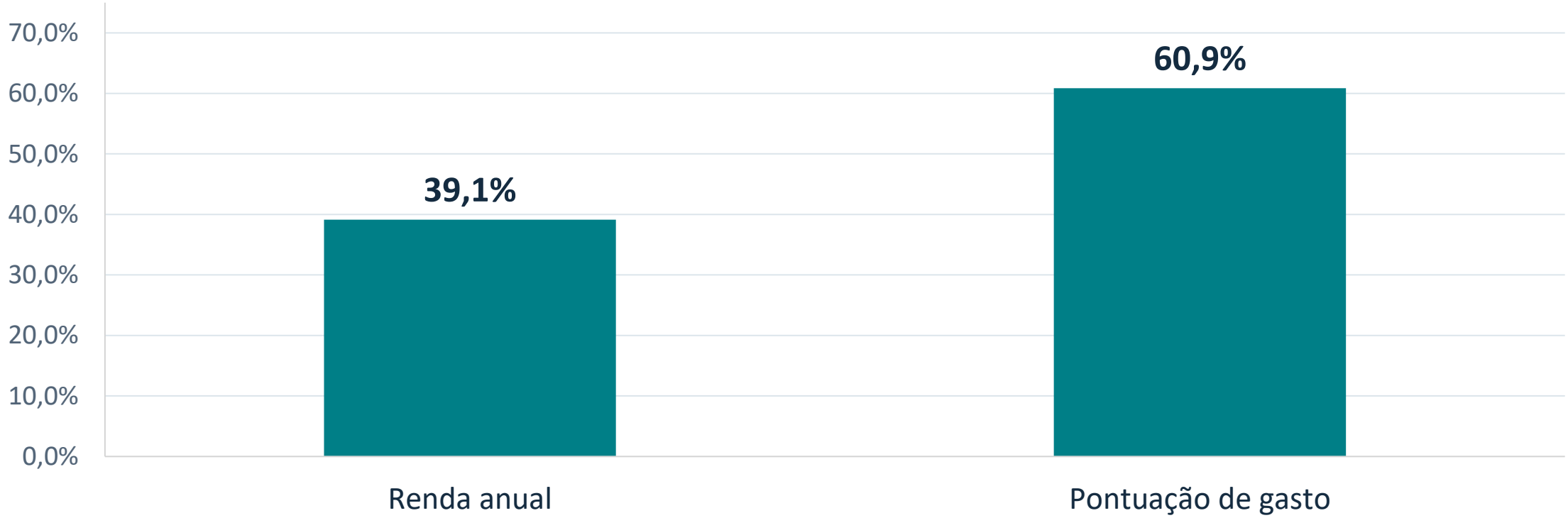
Agregação	Precisão	Recall	F1
Macro	0,9549	0,9250	0,9339
Ponderada	0,9556	0,9500	0,9489

Macro: cada classe tem o mesmo peso.

Ponderada: cada classe pesa conforme seu support.

A classe 0 tem 16 exemplos. A classe 2 tem apenas 4.

Importância dos atributos na árvore



Pontuação: 0,6086. Renda: 0,3914. Importância relativa nesta árvore, sem interpretação causal.

Validação cruzada com cinco partes

Rodada	Parte de validação	Demais partes
1	Parte 1	Treinamento
2	Parte 2	Treinamento
3	Parte 3	Treinamento
4	Parte 4	Treinamento
5	Parte 5	Treinamento

O código calcula a média e o desvio padrão das cinco acurácias.

Resultados salvos da validação cruzada

Modelo	Média de acurácia	Desvio entre partes
Naive Bayes	96,5%	5,83 p.p.
Árvore de decisão	81,0%	20,65 p.p.
Regressão logística	95,5%	4,00 p.p.

A árvore varia mais entre as divisões nesta execução.

Valores do procedimento original, com a ressalva de pré-processamento explicada a seguir.